Scalable Agentic System with PayPal API Integration

Name: Nasgath Farhantha

1. Problem Statement

When a Large Language Model (LLM) is provided with a large number of tools (100+ APIs), its performance degrades. The model may select incorrect tools, generate invalid parameters, or fail to complete tasks. This creates a scalability issue when building real-world agentic systems that must interact with many APIs.

---

2. Proposed Approach

To solve this, a **multi-layer agent architecture** is used. Instead of exposing all tools directly, the system introduces an **intent classifier and tool routing mechanism**.

- The user query is first classified into an intent
- A router selects the relevant domain
- A domain-specific agent handles the task
- Only a small subset of tools is visible at each step

This reduces complexity and improves accuracy.

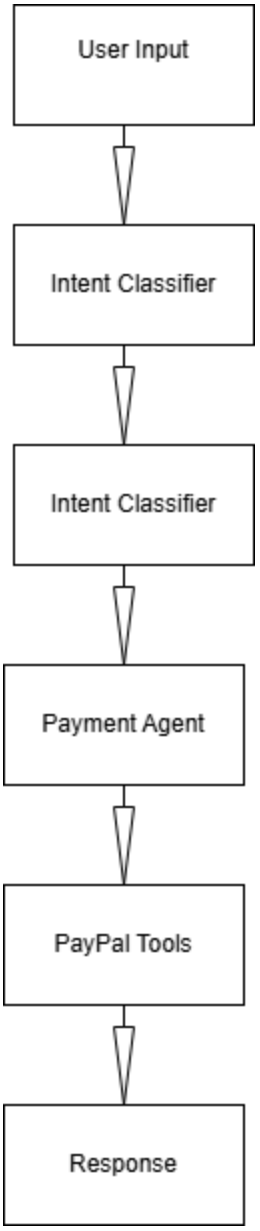
---

3. System Architecture

The system consists of the following components:

- **Intent Classifier:** Identifies user intent
- **Tool Router:** Selects the appropriate domain
- **Domain Agent (Payment Agent):** Handles task execution
- **Tool Executor:** Calls APIs
- **Response Generator:** Returns output

Architecture Diagram



---

4. PayPal API Usage

The PayPal Postman collection is used to implement real API calls. Key APIs such as:

- Create Order
- Capture Payment

are wrapped as tools inside the system.

The agent selects and executes these tools based on user input instead of directly exposing all APIs.

---

5. Example Flow

User: *“Make a payment of $50”*

Steps:

1. Intent classified as Payment
2. Router selects Payment Agent
3. Create Order API is called
4. Capture Payment API is executed
5. Response returned

---

6. Scalability Design

The system handles large numbers of tools using:

- **Tool Grouping:** APIs are grouped into domains
- **Hierarchical Routing:** Multi-step decision making
- **Limited Tool Exposure:** Only relevant tools are used

This ensures the LLM is not overloaded.

---

7. Additional Tools

The system also supports:

- **RAG Tool:** Retrieves information from knowledge base
- **System Search Tool:** Finds available tools and system status

These are treated as separate tools within the architecture.

---

8. Trade-offs

Approach	Advantage	Limitation
Single Agent	Simple	Not scalable
Multi-Agent	Scalable	More complex

Conclusion

The proposed system improves LLM performance by reducing tool complexity through structured routing and abstraction. By integrating PayPal APIs as tools within a hierarchical agentic framework, the system demonstrates a scalable and practical approach for real-world applications.